

HelixConstitution

HelixConstitution

Die universelle Ingenieursverfassung, die jedes Projekt erbt - Anti-Bluff-Gesetz, mechanisch durchgesetzt, als ein Git-Submodul geteilt.

Zusammenfassung

HelixConstitution ist das einzige, projektunabhängige Regelwerk – als Git-Submodul in jedes Helix/vasic-digital-Projekt eingebunden –, das unverhandelbare Ingenieursdisziplin kodifiziert (Anti-Bluff, validierungsbasierte Beweisführung, Datensicherheit, Host-Sicherheit, Dokumentation und Testabdeckung) und an eine Flotte von über 140 Repositories weitergibt. Es bildet das Governance-Rückgrat, das die gesamte Familie kohärent hält.

Kurzbeschreibung

Ein universelles, vererbbares Constitution, ausgeliefert als Git-Submodul. Es definiert verbindliche, unverhandelbare Regeln – Anti-Bluff-Beweisschranken, Immunität gegen falsch-positive Ergebnisse, Datensicherheit, Host-Sicherheit, Testabdeckung und Dokumentationsdisziplin –, die jedes nutzende Projekt automatisch erbt und erweitern, aber niemals abschwächen darf.

Ausführliche Beschreibung

HelixConstitution ist die kanonische, einzige Quelle der Wahrheit für die Ingenieurspraktiken, die in jedem Projekt gelten, das sich durch Einbindung als Git-Submodul anschließt – Ingenieursrecht, verteilt und versionsgebunden wie Code. Sein Kernstück – Constitution.md – ist ein kontinuierlich versioniertes Dokument von etwa 1 MB mit nummerierten Klauseln (die §11.4.x-Kovenenfamilie, aktuell bis §11.4.170) sowie agentenspezifischen Betriebsanleitungen (CLAUDE.md, AGENTS.md, QWEN.md, GEMINI.md), die es per Verweis einbinden, sodass sowohl Menschen als auch jeder

CLI-Agent aus ein und demselben Regelwerk lesen. Die Vererbung erfolgt bewusst in drei Schichten: die universelle Basis (dieses Submodul), die Projektebene (eigenes Constitution/CLAUDE/AGENTS des Projekts, das es erweitert) und eine optionale Unterverzeichnisebene – ausgewertet von oben nach unten, wobei ein Projekt die Regeln *verschärfen*, sie aber *architektonisch niemals abschwächen* darf. Das Ergebnis ist eine Flotte von über 140 Repositories, die nicht stillschweigend auseinanderdriften können, weil die gemeinsame Disziplin versionsgebunden ist, nicht bloß im Gedächtnis verankert.

Das Dokument ist kompromisslos domänenunabhängig: Alles, was einen bestimmten Anbieter, eine Hardware-SKU, einen Port oder eine Bibliotheksversion benennt, muss in das eigene Constitution des nutzenden Projekts verlagert werden, und Universalität wird niemals vorausgesetzt – sie muss durch einen expliziten Vier-Punkte-Test *verdient* werden, bevor eine Regel in die Basis aufgenommen wird. Sein philosophischer Kern ist Anti-Bluff, ausgedrückt als ineinandergreifende Familie von Kovenen – §1.1 Immunität gegen falsch-positive Ergebnisse, §11.4 Endnutzer-Qualitätskovene, §11.4.6 Kein-Raten, §11.4.69 Taxonomie positiver Beweise –, deren kombinierte Wirkung eine einzige harte Linie zieht: Der Maßstab für die Auslieferung ist nie „Tests bestehen“, sondern „ein echter Nutzer kann die Funktion verwenden“, und jedes positive Ergebnis muss auf erfassten physischen Beweisen beruhen, sonst zählt es nicht. Ein begleitender submodules-catalogue.md (142 Repositories) verwandelt die Frage „Haben wir schon etwas, das das kann?“ in einen katalogbasierten Reflex – „Erweitern statt neu implementieren“ –, bevor auch nur eine Zeile neuen Codes geschrieben wird. Hilfsskripte lokalisieren das Submodul aus beliebiger Verschachtelungstiefe und verteilen jeden Commit an vier unabhängige Git-Anbieter, sodass das eine verbindliche Regelwerk auch unmöglich verloren gehen kann.

Warum wir es entwickelt haben

Mehrere große Produkt-Apps und Dutzende entkoppelte, wiederverwendbare Submodule – alle vom selben Eigentümer verantwortet – leiteten immer wieder dieselben mühsam erarbeiteten Regeln neu ab und stießen dabei stets auf dieselbe Art von Fehlern: Tests und Statusberichte, die Erfolg melden, während die Funktion für den Endnutzer defekt ist („PASS-Bluffs“ und „FAIL-Bluffs“). Jeder forensische Anker in den Constitution-Aufzeichnungen dokumentiert einen realen Vorfall (etwa den PASS-Bluff vom 20. Mai 2026 im D3-Audio-Routing, bei dem die Validierung grün anzeigte, obwohl das Feld „Verwendeter Codec“ leer war, oder die Riesen-Schaltfläche vom 25. Juni 2026, die Token-Gleichheitstests bestand, während der eigentliche Bildschirm nicht funktionierte). Der Constitution existiert, um diese ganze Klasse unehrlicher Erfolgsmeldungen ein für alle Mal

mechanisch unmöglich zu machen – damit die Disziplin nicht zwischen Projekten verwässert oder stillschweigend in Vergessenheit gerät.

Warum es ein Game-Changer ist

Es verwandelt die Ingenieurskultur von „Dokumentation, die man hoffentlich befolgt“ in ein vererbtes, versioniertes und mechanisch durchgesetztes Gesetz – der Unterschied zwischen einem Styleguide und einem Compiler. Ein einziges Submodul-Update aktualisiert die Regeln für die gesamte Flotte gleichzeitig, atomar und nachvollziehbar. Ein einzelnes Anti-Bluff-Gebot ist *garantiert* in jedem nutzenden Repository vorhanden, nicht durch Vertrauen, sondern durch Konstruktion: Ein Propagations-Gate durchsucht die gesamte Flotte wortwörtlich nach der Klauselnummer, und ein dazugehöriger Mutationstest beweist, dass das Gate selbst nicht blufft – selbst die Durchsetzung wird also durchgesetzt. Governance hört auf, ein frommer Wunsch auf einem Wiki zu sein, das niemand liest, und wird zu einer überprüfbaren, testbaren Tatsache, auf die man einen CI-Job ansetzen kann.

Was innovativ ist

- **Constitution als Submodul** – Ingenieursrecht wird exakt wie Code verteilt und versioniert, mit bewussten v1.0.0-Tags und projektspezifischen Versionierungen, sodass jedes Repository *genau* weiß, welche Fassung des Regelwerks für es bindend ist.
- **Anti-Bluff als erstklassiges, forensisches Prinzip** – Jede Klausel lässt sich auf eine wortgetreue Anweisung der Verantwortlichen und oft auf den konkreten Vorfall zurückführen, der sie motiviert hat, sodass das Regelbuch wie Fallrecht und nicht wie eine Meinung gelesen wird.
- **Meta-Tests der Regeln selbst (§1.1)** – Jedes Gate wird mit einer Mutation gepaart, die PASS→FAIL auslösen *muss*, sodass „das Gate ist kein Fake“ nicht behauptet, sondern bei jedem Lauf bewiesen wird; ein Gate, das nie fehlschlägt, gilt als schlimmer als gar kein Gate.
- **Erworbene Universalität** – Ein expliziter Vier-Punkte-Test entscheidet, ob eine Regel wirklich universell oder nur projektspezifisch ist, um die Basis schlank, portabel und frei von Anbieterabhängigkeiten zu halten.

Wie es in allen Produkten eingesetzt wird (die Möglichkeiten, die es bietet)

Als **verpflichtende Governance-Säule** ist HelixConstitution kein Dokument, das die Familie konsultiert – es ist das tragende Gerüst, auf dem die Familie aufgebaut ist:

- **Governance-Rückgrat:** Jedes Helix/vasic-digital-Projekt fügt es als Submodul hinzu und importiert es aus CLAUDE.md / AGENTS.md / QWEN.md oder der eigenen Constitution.md; die Regeln gelten bedingungslos ab dem ersten Commit, ohne projektspezifische Ausnahmen.
- **Gates & Vorgaben:** Es definiert das Vier-Ebenen-Abdeckungsmodell – Quellcode vorhanden, überlebt den Build, verhält sich zur Laufzeit korrekt, Gate blufft nicht –, das ein Feature auf allen vier Ebenen bestehen muss, bevor es als „fertig“ gilt, sowie eine wachsende Liste benannter Vorgaben: Umgang mit Credentials (§11.4.10), stets synchronisierte Dokumentation (§11.4.60), Containers-Submodul-Vorgabe (§11.4.76), CodeGraph (§11.4.78), verpflichtende Testtyp-Abdeckung (§11.4.169) und mehr.
- **Propagation:** CM-COVENANT-114-NNN-PROPAGATION-Gates prüfen, ob der *wörtliche* Klauseltext in der gesamten nutzenden Flotte vorhanden ist, sodass eine Vorgabe nicht stillschweigend in einer Ecke des Systems fallen gelassen werden kann; Nichteinhaltung ist ein harter Release-Blocker ohne Schlupflöcher.
- **Entdeckung:** submodules-catalogue.md macht die Frage „Haben wir schon etwas, das X kann?“ mit einem Blick beantwortbar, bevor ein neues Modul aufgesetzt wird – und verhindert so Doppelarbeit an der Wurzel.
- **Konsistenz der AI-Agenten:** Dasselbe Regelwerk wird identisch an jeden CLI-Agenten übermittelt (Claude Code, Codex/Cursor/Aider/OpenCode/Crush/Kimi über AGENTS.md, Qwen Code über QWEN.md), sodass unabhängig davon, welches Tool den Code berührt, es ein und demselben Vertrag gehorcht.

Größte technische Herausforderungen & wie wir sie gelöst haben

- **Submodul aus beliebiger Verschachtelungstiefe lokalisieren** – eine Regel, die drei Submodule tief vergraben ist, muss das Gesetz trotzdem finden, ohne zu wissen, wo es sich befindet → find_constitution.sh durchläuft übergeordnete Verzeichnisse und folgt rekursiv dem Git-Superprojekt-Zeiger, berücksichtigt eine CONSTITUTION_DIR-Überschreibung und zwei unterstützte Verzeichnisstrukturen (constitution/, submodules/

constitution/), sodass die Auflösung unabhängig von der Verschachtelungstiefe deterministisch bleibt.

- **Ein Repository als maßgebliche Quelle über vier Git-Anbieter hinweg verwalten** – Spiegel sind wertlos, wenn sie auseinanderdriften → `install_upstreams.sh` liest deklarative `Upstreams/*.sh`-Remotes ein und konfiguriert `origin` mit mehreren Push-URLs, sodass ein einziger `git push` atomar an GitHub (Primärquelle), GitLab, GitFlic und GitVerse verteilt wird und kein Spiegel zurückfällt.
- **Regel-Wucherung / Projektspezifische Inhalte in der universellen Basis verhindern** – jede verlockende „einfach hier hinzufügen“-Lösung untergräbt die Portabilität → der Vier-Punkte-Test auf „verdiente Universalität“ plus die Klassifizierung nach §11.4.17 (universell vs. projektspezifisch) wird auf *jede* neue Regel angewendet, um projektspezifische Anliegen dorthin zurückzudrängen, wo sie hingehören: in die Projektschicht.
- **Nachweis, dass das Vererbungs-Gate tatsächlich funktioniert** – ein Gate, das nie versagt, ist ein Gate, dem man nicht trauen kann → `meta_test_inheritance.sh`, ein Wächter-Metatest, löscht gezielt den §11.4-Anker und prüft, ob das Gate dies erkennt, sodass der Durchsetzungsmechanismus selbst kontinuierlich gegen stille Ausfälle überprüft wird.

Technologie-Stack

- **Git-Submodul-Vererbung** – *warum*: Git-Submodule sind der einzige Mechanismus, der es ermöglicht, dass ein Regelwerk *maßgeblich* bleibt und gleichzeitig für jeden Nutzer versionsgebunden ist, aktualisiert durch einen expliziten, prüfbaren Schritt statt durch stummes Kopieren und Einfügen; *wie*: nutzende Projekte fügen das Submodul hinzu und importieren dessen Agenten-Dateien mit `@import`, wobei die drei Ebenen von oben nach unten ausgewertet werden – mit einem strikten „Erweitern, nicht Abschwächen“-Vertrag an jeder Schnittstelle.
- **find_constitution.sh** – *warum*: Regeln sind nutzlos, wenn tief verschachtelter Code sie nicht zuverlässig finden kann, und hartcodierte Pfade würden bei jeder Projektumstrukturierung brechen; *wie*: ein Durchlauf über übergeordnete Verzeichnisse plus rekursive Abfrage von `git rev-parse --show-superproject-working-tree`, abgesichert durch eine `CONSTITUTION_DIR`-Überschreibung, die beide unterstützten Verzeichnisstrukturen auflöst.
- **install_upstreams.sh + Upstreams/** – *warum*: Vier-Anbieter-Redundanz ist nur real, wenn sie keinen zusätzlichen Wartungsaufwand verursacht, sonst veralten die Spiegel; *wie*: deklarative `.sh`-Dateien pro Remote werden in ein einziges Multi-URL-origin überführt, sodass vier Push-Vorgänge zu einem zusammengefasst werden.

- **§1.1-Mutationstests** – *warum*: Ein Gate, das nie versagt, ist schlimmer als keins, weil es falsche Sicherheit vorgaukelt; *wie*: Jedes Gate wird mit einer sed-basierten Lösch-/Umbenennungsmutation gepaart, die zwingend von PASS auf FAIL wechseln muss, bevor sie wiederhergestellt wird – sodass jedes Gate bei jedem Durchlauf beweist, dass es noch „zubeißt“.
- **Verbreitungs-Gates (CM-COVENANT-114-NNN-PROPAGATION)** – *warum*: Ein Vertrag ist nur dann universell, wenn er nachweislich in *jedem* Nutzerprojekt vorhanden ist, nicht nur im Flagship-Repository; *wie*: ein wörtlicher Paragrafen-Nummern-Grep über alle Nutzerprojekte, gestützt durch einen §1.1-Mutationstest, der beweist, dass die Verbreitungsprüfung selbst fehlschlagen kann.
- **submodules-catalogue.md (§11.4.74)** – *warum*: Der schnellste Weg, die Disziplin gegen Duplikate zu verletzen, ist, nicht zu wissen, was man bereits besitzt; *wie*: ein 142-Repository umfassender, nach Fähigkeiten gruppierter Katalog, dessen Prüfung im Tracker *vor* der Einrichtung neuer Strukturen dokumentiert wird.
- **Mehrformat-Export** – *warum*: Dasselbe Gesetz muss gleichermaßen für Menschen lesbar, von Tools parsbar und in Archiven bewahrbar sein; *wie*: Jedes kanonische Dokument wird aus einer Quelle in .md / .html / .pdf / .docx ausgegeben.

Status & Ehrlichkeitshinweise

- **Status: ausgeliefert.** Wird aktiv versioniert und als Submodul im gesamten Verbund genutzt (öffentliche kanonische und Spiegel-Repositories).
- **Lizenz: noch zu klären** – im geprüften Quellmaterial nicht explizit angegeben; vor der Veröffentlichung mit der Repository-LIZENZDATEI abgleichen.
- Weitere Upstream-Spiegel: GitLab helixdevelopment1/helixconstitution, GitFlic helixdevelopment/helixconstitution, GitVerse helixdevelopment/HelixConstitution.

Prioritätsstufe: Helix-primär — ein verbindlicher Governance-Pfeiler, auf dem alles in der Helix-Familie aufbaut.